

03 — Building Worlds

World-building in MegaMOO happens **from inside the game**. A builder logs in, walks to where they want to create something, and types @-prefixed commands that create rooms, carve exits, make objects, and program behavior. These commands are themselves verbs (on #3, `Base_Character`), built on the primitives described in Writing Verbs — so the toolkit is just more MegaMOO, and you can extend it the same way.

This document is a reference to the in-game builder toolkit — around sixty commands — plus a worked example.

- [Permissions](#)
- [A worked example](#)
- [Rooms and exits](#)
- [Objects: create, name, describe](#)
- [Properties and tags](#)
- [Exit and action messages](#)
- [Programming verbs](#)
- [Inspection and debugging](#)
- [Colors and message tokens](#)
- [Movement and structure](#)
- [Command reference](#)

Permissions

Every builder command requires a minimum permission level. The ladder runs `gm1` (AssistantGM) → `gm2` (Builder) → `gm3` (Coder) → `gm4` (Admin) → `gm5` (God); see Operations for how levels are granted and how they map to the `PROGRAMMER` / `WIZARD` flags. As a rough guide:

- **gm2 (Builder):** dig rooms, open exits, make and describe objects, set messages. Everything needed to author content without writing code.
- **gm3 (Coder):** create objects from arbitrary parents, add/remove properties, program verbs, reparent, inspect internals, evaluate code.
- **gm4–gm5:** player administration, renumbering, number reservation, and server control.

Throughout this document each command notes its minimum level.

A worked example

Build a small two-room area with a connecting path and an item:

```
# 1. Dig an in-character room and teleport into it
@dig/tel ic = Mossy Clearing
@desc here = Sunlight filters through the canopy onto a carpet of soft moss.

# 2. Dig a second room (don't teleport yet)
@dig ic = Hollow Log

# ...note its object number from the output, say #142, then connect them:
@gopen path to #142          # named walkable exit, with an auto return exit
@vopen north to #142         # OR a virtual compass exit, if you prefer N/S/E/W

# 3. Make an item directly in the room (/drop), then dress it up
@make/drop #12 = lantern      # child of #12 (item); /drop places it here, not in
inventory
@desc lantern = A battered brass lantern, its glass smudged with soot.
@adjective lantern = battered brass

# 4. Inspect what you built
@examine here
+show lantern
```

Two stylistic choices appear here and recur throughout the toolkit:

- **Exits come in flavors** — directional vs. named, virtual vs. object-backed, plain vs. closable/climbable/jumpable. Pick the lightest one that fits.
- **Object identity is parent = name** — `@make <parent> = <name>` mirrors the `@dig <roomtype> = <roomname>` shape. The parent determines inherited behavior.

Rooms and exits

Rooms

Command	Level	What it does
<code>@dig[/types][/tel]</code> <code><roomtype> [= <name>]</code>	gm2	Create a room of <code>roomtype</code> (<code>/types</code> lists available types). <code>/tel</code> teleports you into it. Name defaults to the parent's noun.

`roomtype` selects the parent class — typically `ic` (#17 ICRoom) or `ooc` (#16 OCRoom).

Exits

MegaMOO distinguishes several exit kinds so movement reads naturally and so the engine can model doors, climbing, and jumping:

Command	Level	Creates	Use for
<code>@open <dir> [to <dest>]</code>	gm2	An object-backed directional exit (#21)	A compass exit you'll customize per-exit.
<code>@vopen <dir> to <dest></code>	gm2	A virtual directional exit stored in the room's <code>dexits</code>	Standard N/S/E/W links; lightest weight.
<code>@gopen[/noret] <name> [to <dest>]</code>	gm2	A named walkable passage (#22 GoExit)	"path", "archway", "trail".
<code>@dopen[/noret] <name> to <dest></code>	gm2	A closable door (#23) — starts closed	"gate", "door", "hatch".
<code>@copen[/noret] <name> [to <dest>]</code>	gm2	A climbable exit (#24)	"slope", "ladder", "wall".
<code>@jopen[/noret] <name> [to <dest>]</code>	gm2	A jumpable exit (#25)	"gap", "chasm", "ledge".
<code>@rmexit <exit></code>	gm2	—	Removes an exit (and its matching return exit).
<code>@virtualize <exit></code>	gm2	—	Converts an object-backed directional exit into a virtual one and recycles the original.

By default the `@*open` commands also create a **return exit** in the destination room; pass `/noret` to suppress that (one-way exits). Valid directions are `north south east west ne nw se sw u d o in` — see the full compass table and reverse mappings (note `out / in` have no automatic reverse).

Virtual vs. object-backed exits. A virtual exit is just data in the room's `dexits` property — cheap, perfect for ordinary compass links. An object-backed exit is a real object you can give custom verbs, messages, locks, and keys. Start virtual; `@open / @gopen` when you need behavior.

Objects: create, name, describe

Command	Level	What it does
<code>@make[/drop] <parent> = <name> [with <owner>]</code>	gm3	Create an object as a child of <code>parent</code> (by <code>#num</code> , <code>\$const</code> , or name). <code>/drop</code> puts it in the room instead of your inventory.
<code>@delete <object></code>	gm2	Permanently recycle an object (confirms first; core objects #0–#9 are protected).
<code>@name <object> = <name></code>	gm2	Set the noun and auto-detect the article (capitalized → proper noun, no article).
<code>@desc <object> = <text></code>	gm2	Set the description (what <code>look</code> shows). <code>@desc x =</code> clears it.
<code>@adjective <object> = <a1> [a2] [a3]</code>	gm2	Set up to three adjectives (matched by <code>pmatch</code>). Clear with <code>@adjective x =</code> .
<code>@article <object> = <a/an/the/some></code>	gm2	Override the article.
<code>@trailer <object> = <text></code>	gm2	Set trailing text in the title (“a sword <i>with a golden hilt</i> ”).
<code>@title <object></code>	gm2	Regenerate the display title from noun + modifiers.

An object’s display name is composed from `article + adjectives + noun + trailer`, which is why these are separate commands — each piece participates in matching and in how the name renders in different grammatical positions.

Properties and tags

Command	Level	What it does
<code>@adprop <object>.<prop> [= <value>]</code>	gm3	Add a new property. The value is evaluated as a Python expression first (<code>[]</code> , <code>0</code> , <code>{}</code>); if that fails it's stored as a string.
<code>@set <object>.<prop> [= <value>] (@val)</code>	gm3	Set an existing property (local or inherited), or read it if <code>= <value></code> is omitted. Same literal-then-string evaluation as <code>@adprop</code> . Records the previous value.
<code>@unset <object>.<prop></code>	gm3	Revert the last <code>@set</code> on that property.
<code>@rmprop <object>.<prop> (@remprop)</code>	gm3	Remove a locally defined property (confirms).
<code>@clear[/all] <object>[.<prop\ verb>]</code>	gm3	Clear one local property/verb, or with <code>/all</code> every local property.
<code>@adtag <object> = <category>[/<tag>]</code>	gm3	Add a classification tag (e.g. <code>zone/haven</code>).
<code>@rmtag <object> = <category>[/<tag>]</code>	gm3	Remove a tag, or an entire category if no <code>/tag</code> given.

`@adprop here.spawns = []` then editing it from a verb is the typical way to add custom per-object state that game logic reads later.

Exit and action messages

Exits (and other usable objects) carry message pairs: one shown to the actor, one (`o` -prefixed) shown to onlookers, with emit tokens like `%S` .

Command	Level	Shown to	When
@success <obj> = <msg>	gm2	actor	On successful use (“You head north.”)
@osuccess <obj> = <msg>	gm2	others in the source room	“%S heads north.”
@failure <obj> = <msg>	gm2	actor	On failed use (“The door is locked.”)
@ofailure <obj> = <msg>	gm2	others	“%S struggles with the door.”
@drop <obj> = <msg>	gm2	actor on arrival	“You arrive at the clearing.”
@odrop <obj> = <msg>	gm2	others in the destination room	“%S arrives.”

Clearing any of them is @<cmd> <obj> = with an empty right-hand side. See [message tokens](#) for the substitution vocabulary.

Programming verbs

This is where building shades into coding. Verbs can be created and edited entirely in-game *or* on disk, and the two stay in sync.

Command	Level	What it does
<code>@adverb[/hidden]</code> <code><obj>.<name[(min)]</code> <code>[,alias...]> [with <perms></code> <code>[base] [min=N] [auth=N]]</code>	gm3	Add a verb. Names can carry a minimum-abbreviation length in parens; commas add aliases. <code>perms</code> defaults to <code>rx</code> ; <code>auth=N</code> sets the minimum gm level to invoke; <code>/hidden</code> makes it non-invokable by players (for hooks).
<code>@program <obj>.<verb></code>	gm3	Open the in-MOO line editor for a verb. On save it compiles, installs the verb into the live server immediately, and writes the source to disk — see Hot coding below.
<code>@reload <obj>.<verb> /</code> <code>@reload <obj> / @reload</code> <code>all</code>	gm3	Force a pull of verb source from disk into the running server. Rarely needed — the auto-reload watcher normally does this for you (see below). On first use for an object it creates the per-object directory and exports existing verbs; <code>@reload all</code> scans every directory and loads/creates/updates.
<code>@rmverb <obj>.<verb></code>	gm3	Remove a locally defined verb (confirms).
<code>@adalias <obj>.<verb> =</code> <code><alias></code>	gm3	Add another name to an existing verb. Refuses a name already used by another verb on that object.
<code>@rmalias <obj>.<verb> =</code> <code><alias></code>	gm3	Drop one alias from a verb, along with its <code>@min</code> setting. Refuses to remove the primary (first) name — use <code>@rmverb</code> for that.
<code>@min <obj>.<verb> = <N></code>	gm3	Set how many characters a player must type to match the verb.
<code>@hideverb <obj>.<verb></code>	gm3	Hide a verb (e.g. an internal hook like <code>at_post_move</code>).
<code>@unhideverb <obj>.<verb></code>	gm3	Un-hide a hidden verb, making it player-invokable again.
<code>@verbauth <obj>.<verb> [=</code> <code><level>]</code>	gm3	View or set the minimum auth level required to invoke a verb.

Hot coding, no reloads

Editing a verb with `@program` is **hot** — there is no separate reload or restart step. When you finish the editor (`.` on a line of its own), the engine compiles the code, and on success **installs it into the live database right then**: an existing verb’s bytecode is recompiled in place, a new verb is added to the object. The change is active on the *next* invocation of the verb. `@program` *also* writes the source to `moo verbs/<objnum>/<verbname>.py`, so the live world and the on-disk copy never drift. If the code has a syntax error, nothing is installed and nothing is saved — the live verb keeps running.

Disk edits are hot too: the auto-reload watcher

The disk direction is also automatic. The server runs a background watcher (`dev.autoreload_verbs`, **on by default**, polling every `dev.autoreload_interval` = 2 seconds) that compares file mtimes under `moo verbs/` and re-loads any verb whose file changed. Edit `moo verbs/17/wave.py` in a normal editor, save, and the change is live a couple of seconds later — no command, no restart.

`@reload` is the manual version of that same disk→database pull. You need it only when you don’t want to wait for the poll, or when the watcher is disabled in config. You do *not* run it after

`@program` .

The two editing loops

In-game (hot):

1. `@program #17.wave` — type the code, end with `.` .
2. Done. It’s compiled, live, and written to disk in one step.

On disk (also hot):

1. `@adverb #17.wave` to declare the verb if it doesn’t exist yet.
2. Edit `moo verbs/17/wave.py` in a normal editor — or via the MCP integration, which exposes `get_verb` / `set_verb` to an AI assistant.
3. Save. The watcher picks it up within ~2 seconds. (`@reload #17.wave` if you want it *now*.)

Because verb source is mirrored to plain files, the whole world’s behavior is diff-able and version-controlled — while remaining editable live from inside the game, with no build or restart in either loop.

What is *not* hot: engine Python under `moo/` — builtins, verb types, the parser, the namespace builder. Those changes need `@restart` .

Inspection and debugging

Command	Level	What it shows
@examine <object>	gm3	Parent, owner, location, flags, children, contents, all local properties (values truncated), and local verbs with line counts.
+show <object>	gm1	Noun/name/id, the full parent chain and location chain, owner, flags, children, contents.
+props[/all] <object>	gm3	Local property names (or, with /all , the whole inheritance chain grouped by object).
+verbs[/all] <object>	gm3	Local verb names (or the full chain); * marks the minimum-abbreviation point.
+decompile <object>.<verb>	gm3	Print a verb's source (requires r permission on the verb).
@list [<start>] [to <end>]	gm3	Objects in a number range, with parent and name.
eval <expr> / / <expr>	gm3	Evaluate arbitrary Python in the verb context. / len(db.objects()) , / pobj.objnum .
+pron	gm2	Reference table of pronoun substitution tokens.
@color	gm1	Reference table of color codes.

me and here are valid object references everywhere (+show here , @examine me), as are #N and \$name .

Colors and message tokens

Color codes

Output codes are %-prefixed and translated to ANSI:

Code	Effect
<code>&r &g &b &y &m &c &w &k</code>	red / green / blue / yellow / magenta / cyan / white / black
<code>&R &G &B ...</code> (uppercase)	bold/bright variants
<code>&<245></code>	xterm-256 color (245 = dim gray, good for UI chrome)
<code>&<#FF0000></code>	hex RGB
<code>&n</code>	reset

Conventions: `&<245>` for dim chrome, avoid `&<240>` (too dark). **All color is stripped under screenreader mode**, so never rely on color alone to convey meaning. `@color` prints the live reference.

Emit tokens

The same tokens from Writing Verbs apply in builder messages (`@osuccess`, etc.). Both `%` and `$` prefixes work:

Token	Renders
<code>%S / %s</code>	subject's capitalized / plain name
<code>%D / %d</code>	direct object
<code>%ps %po %pp %pa %pr</code>	subject's gender pronouns

`+pron` prints the live reference.

Movement and structure

Command	Level	What it does
<code>@tel <dest> / @telq <dest></code>	gm1	Teleport to <code>#N</code> , <code>home</code> , <code>mark</code> , or a character's location. <code>@telq</code> is silent (no departure/arrival messages). Won't cross the IC/OOC boundary.
<code>@move <obj> to <dest> [with <msg>]</code>	gm2	Move an object to a room/container, with an optional arrival message.
<code>@parent <obj> = <new_parent> (or a #start to #end range)</code>	gm3	Reparent one object or a range.

Game-specific staff commands. The `moo verbs/3/` directory also contains commands that exist only to drive a particular game's systems (character progression, body/equipment

models, creature spawning, and the like). Those are part of the game built on the engine, not the engine itself, and are documented with the game rather than here.

Command reference

The full toolkit grouped by purpose. Minimum level in parentheses.

Rooms & exits: @dig (2), @open (2), @vopen (2), @gopen (2), @dopen (2), @copen (2), @jopen (2), @rmexit (2), @virtualize (2)

Objects: @make (3), @delete (2), @name (2), @desc (2), @adjective (2), @article (2), @trailer (2), @title (2)

Properties & tags: @adprop (3), @set / @val (3), @unset (3), @rmprop / @remprop (3), @clear (3), @adtag (3), @rmtag (3)

Messages: @success (2), @osuccess (2), @failure (2), @ofailure (2), @drop (2), @odrop (2)

Verb programming: @adverb (3), @program / @code / @prog (3), @reload (3), @rmverb (3), @adalias (3), @rmalias (3), @min (3), @hideverb (3), @unhideverb (3), @verbauth (3)

Inspection: @examine (3), @list (3), +show (1), +props (3), +verbs (3), +decompile (3), eval / / (3), +pron (2), @color (1)

Structure & movement: @parent (3), @move (2), @tel / @telq (1)

Player admin: @auth (5), @delplayer (4)

Database & server: @renumber (5), @reserve (5), @freeon / @freeoff (3), @shutdown (5), @restart (5)

The player-admin and server commands are covered in Operations.

Next: Operations — running the server, configuration, the JSON API, and the MCP integration.